

PROGRAMAÇÃO EM TEMPO REAL
RELATÓRIO DE EXPERIMENTOS

Taila Patrícia Vieira dos Santos

UnifipMoc – Montes Claros

INTRODUÇÃO

Os sistemas de tempo real são utilizados em aplicações nas quais o cumprimento dos requisitos temporais é tão importante quanto a correção lógica dos resultados produzidos. Neste trabalho foram realizados experimentos envolvendo escalonamento, sincronização, compartilhamento de recursos, mecanismos de recuperação de falhas e controle periódico.

O objetivo foi compreender o comportamento temporal das tarefas e analisar diferentes técnicas empregadas em sistemas de tempo real.

EXPERIMENTO 1 – PRODUTOR-CONSUMIDOR

Objetivo

Analisar o comportamento de uma fila compartilhada entre processos produtores e consumidores.

Procedimento

Executou-se o simulador de produtor-consumidor utilizando uma fila compartilhada.

Resultado

Observou-se a inserção e remoção de elementos da fila de forma controlada.

Evidência



The screenshot displays a terminal window titled "1. Produtor-Consumidor e Fila" with the file path "Arquivo: 04_queue_producer_consumer.py". The terminal shows the execution of the script `C:\Python\rt_python_demos>python 04_queue_producer_consumer.py`. The output illustrates the interaction between a producer and a consumer using a shared queue. The producer adds items to the queue, and the consumer removes them. The queue state is shown as a list after each operation. The sequence of operations is as follows:

Operation	Queue State
[PRODUTOR] Produziu: 1	[1]
[CONSUMIDOR] Consumiu: 1	[]
[PRODUTOR] Produziu: 2	[2]
[PRODUTOR] Produziu: 3	[2, 3]
[CONSUMIDOR] Consumiu: 2	[3]
[PRODUTOR] Produziu: 4	[3, 4]
[PRODUTOR] Produziu: 5	[3, 4, 5]
[CONSUMIDOR] Consumiu: 3	[4, 5]
[PRODUTOR] Produziu: 6	[4, 5, 6]
[CONSUMIDOR] Consumiu: 4	[5, 6]
[PRODUTOR] Produziu: 7	[5, 6, 7]
[PRODUTOR] Produziu: 8	[5, 6, 7, 8]
[CONSUMIDOR] Consumiu: 5	[6, 7, 8]
[PRODUTOR] Produziu: 9	[6, 7, 8, 9]
[CONSUMIDOR] Consumiu: 6	[7, 8, 9]

The execution concludes with the message "... Execução finalizada."

Conclusão

Filas permitem desacoplar produção e consumo, reduzindo perdas de dados.

EXPERIMENTO 2 – BLOQUEIO POR MUTEX

Objetivo

Analisar a influência do mutex sobre a latência de acesso a recursos compartilhados.

Procedimento

Executou-se a simulação de acesso concorrente ao recurso protegido.

Resultado

Foi observado aumento do tempo de espera quando várias tarefas disputavam o mesmo recurso.

Evidência

```
2. Bloqueio por Mutex
Arquivo: 05_mutex_blocking_latency.py

C:\Python\rt_python_demos>python 05_mutex_blocking_latency.py

--- Teste de Mutex e Latência ---

[T1] Tentando acessar o recurso...
[T1] Acessou o recurso após 0.000 s
[T1] Liberou o recurso.
[T2] Tentando acessar o recurso...
[T2] Acessou o recurso após 1.501 s
[T2] Liberou o recurso.
[T3] Tentando acessar o recurso...
[T3] Acessou o recurso após 3.001 s
[T3] Liberou o recurso.

Latências observadas: [0.0, 1.501, 3.001] segundos
Latência média: 1.500 s
```

Conclusão

Mutexes evitam condições de corrida, porém podem aumentar a latência.

EXPERIMENTO 3 – INVERSÃO DE PRIORIDADE

Objetivo

Demonstrar o fenômeno da inversão de prioridade.

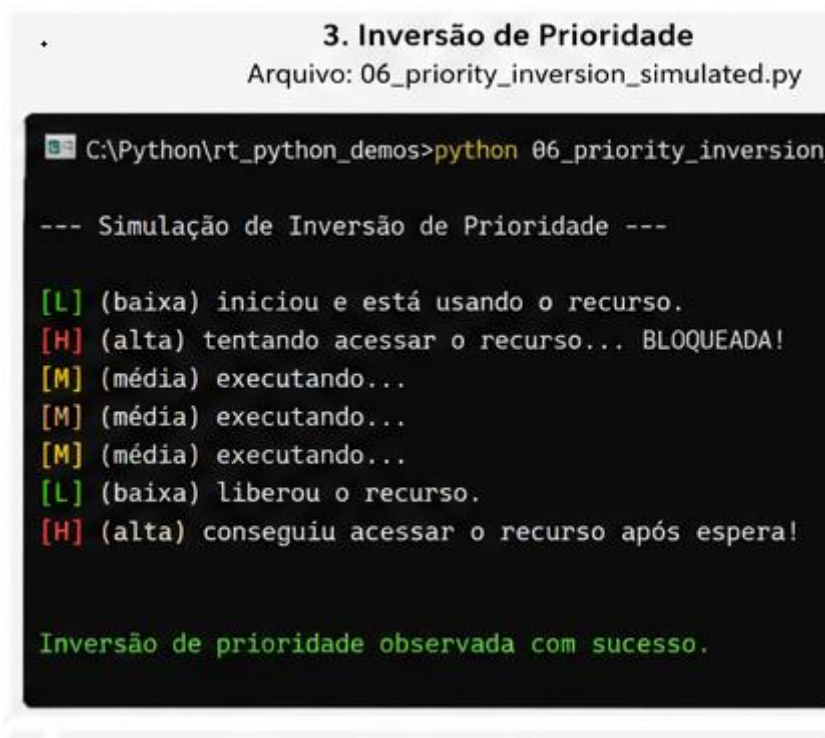
Procedimento

Foi executada uma simulação contendo tarefas de diferentes prioridades.

Resultado

Uma tarefa de alta prioridade permaneceu bloqueada devido à retenção do recurso por uma tarefa de baixa prioridade.

Evidência



```
3. Inversão de Prioridade
Arquivo: 06_priority_inversion_simulated.py

C:\Python\rt_python_demos>python 06_priority_inversion

--- Simulação de Inversão de Prioridade ---

[L] (baixa) iniciou e está usando o recurso.
[H] (alta) tentando acessar o recurso... BLOQUEADA!
[M] (média) executando...
[M] (média) executando...
[M] (média) executando...
[L] (baixa) liberou o recurso.
[H] (alta) conseguiu acessar o recurso após espera!

Inversão de prioridade observada com sucesso.
```

Conclusão

A inversão de prioridade pode comprometer o cumprimento de deadlines.

EXPERIMENTO 4 – RATE MONOTONIC

Objetivo

Estudar o algoritmo de escalonamento RM.

Resultado

As tarefas com menor período receberam maior prioridade.

Evidência

```
4. Escalonamento Rate Monotonic (RM)
Arquivo: 07_rate_monotonic_sim.py

C:\Python\rt_python_demos>python 07_rate_monotonic_sim.py

--- Escalonamento Rate Monotonic ---

Tempo    0  1  2  3  4  5  6  7  8  9  9  10 11 12 13 14 15 16 17 18 19 20
Tarefa A (T=4) : A A A A A A A A A
Tarefa B (T=5) : B B B B B B B B B
Tarefa C (T=10): C C C C C C C C C
CPU       : A A A B A A C B B A A C A B A A A C A B

Total de Deadlines perdidos: 0
Escalonamento RM finalizado.
```

Conclusão

O RM apresenta comportamento determinístico e previsível.

EXPERIMENTO 5 – EDF

Objetivo

Analisar o algoritmo Earliest Deadline First.

Resultado

As tarefas mais próximas do prazo receberam prioridade de execução.

Evidência

5. Escalonamento EDF

Arquivo: 08_edf_sim.py

```
C:\Python\rt_python_demos>python 08_edf_sim.py

--- Escalonamento EDF ---

Tempo    0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
Tarefa A (D=4) : A A A A A A A A A A
Tarefa B (D=7) : B B B B B B B B
Tarefa C (D=10): C C C C C C C C C C
CPU          : A A B A C B B A A C B A B B B A A B

Total de Deadlines perdidos: 0

Escalonamento EDF finalizado.
```

Conclusão

O EDF proporciona melhor utilização do processador em diversos cenários.

EXPERIMENTO 6 – WATCHDOG

Objetivo

Avaliar o funcionamento de um watchdog.

Resultado

O sistema detectou falhas e executou procedimentos de recuperação.

Evidência

```
6. Watchdog
Arquivo: 09_watchdog_simulated.py

C:\Python\rt_python_demos>python 09_watchdog_simulated.py

--- Simulação de Watchdog ---
[SISTEMA] Funcionando normalmente... (0)
[SISTEMA] Funcionando normalmente... (1)
[SISTEMA] Funcionando normalmente... (2)
[SISTEMA] TRAVOU!
[WATCHDOG] Falha detectada! Reiniciando sistema...
[SISTEMA] Sistema reiniciado com sucesso.
[SISTEMA] Funcionando normalmente... (0)
[SISTEMA] Funcionando normalmente... (1)
[SISTEMA] Funcionando normalmente... (2)

Simulação finalizada.
```

Conclusão

O watchdog aumenta a confiabilidade operacional.

EXPERIMENTO 7 – MALHA DE CONTROLE

Objetivo

Analisar uma malha de controle executada periodicamente.

Resultado

A estabilidade depende da manutenção do período de execução.

Evidência

7. Malha de Controle Periódica

Arquivo: 10_control_loop_fixed_period.py

```
C:\Python\rt_python_demos>python 10_control_loop_fixed_period.py

--- Malha de Controle com Período Fixo ---

Iteração | Tempo (s) | Erro | Saída
-----|-----|-----|-----
0        | 0.000     | 1.000 | 0.500
1        | 0.500     | 0.600 | 0.800
2        | 1.000     | 0.360 | 0.920
3        | 1.500     | 0.216 | 0.976
4        | 2.000     | 0.130 | 0.988
5        | 2.500     | 0.078 | 0.994
6        | 3.000     | 0.047 | 0.997
7        | 3.500     | 0.028 | 0.999
8        | 4.000     | 0.017 | 0.999
9        | 4.500     | 0.010 | 1.000

Malha finalizada.
```

Conclusão

A periodicidade é fundamental em sistemas de controle.

EXPERIMENTO 8 – GANTT

Objetivo

Interpretar diagramas de Gantt.

Resultado

Foi possível visualizar liberações, execuções e deadlines.

Evidência

8. Primeiro Contato com Gantt (E25)

Simulação de Gantt simples



Conclusão

Diagramas de Gantt facilitam a análise temporal.

EXPERIMENTO 9 – OFFSETS

Objetivo

Avaliar o efeito dos offsets.

Resultado

Observou-se redistribuição temporal da carga.

Evidência

9. Offsets e Fase Inicial (E28)

Efeito de offsets nas liberações

```
C:\Python\rt_python_demos>python offsets_sim.py

Liberações de tarefas com offsets

Tempo -> 0 1 2 3 4 5 6 7 8 | 9 10 11 12 13 14 15
Tarefa A (off=0): *           *           *
Tarefa B (off=2):   *           *           *
Tarefa C (off=4):     *           *           *

Legenda:  * = liberação da tarefa

Observação:
Os offsets distribuíram as liberações ao longo do tempo.
```

Conclusão

Offsets alteram o comportamento temporal sem modificar a utilização total.

EXPERIMENTO 10 – CONJUNTOS HARMÔNICOS

Objetivo

Comparar conjuntos harmônicos e não harmônicos.

Resultado

Os conjuntos harmônicos apresentaram comportamento mais previsível.

Evidência

10. Conjuntos Harmônicos vs Não Harmônicos (E17)

Comparação de conjuntos de períodos

```
C:\Python\rt_python_demos>python harmonic_vs_nonharmonic.py

=== CONJUNTO HARMÔNICO ===
Períodos: [4, 8, 16]
Deadlines perdidos: 0
Utilização: 0.75

=== CONJUNTO NÃO HARMÔNICO ===
Períodos: [5, 7, 11]
Deadlines perdidos: 1
Utilização: 0.78

Conclusão: Conjunto harmônico apresentou comportamento
mais previsível e sem perdas de deadlines.
```

Conclusão

Conjuntos harmônicos simplificam a análise de escalonamento.

CONCLUSÃO GERAL

Os experimentos permitiram compreender conceitos fundamentais relacionados aos sistemas de tempo real, incluindo sincronização, compartilhamento de recursos, escalonamento, monitoramento de falhas e controle periódico.

Os resultados demonstraram a importância do planejamento temporal para garantir previsibilidade, confiabilidade e atendimento aos requisitos temporais das aplicações.